

中山大學
SUN YAT-SEN UNIVERSITY

并行程序设计与算法实验

Lab1-基于 MPI 的并行矩阵乘法

姓 名 林宏宇

学 号 23320093

学 院 计算机学院

专 业 计算机科学与技术

2025 年 3 月 26 日

1 实验目的

- 掌握 MPI 程序的编译和运行方法。
- 理解 MPI 点对点通信的基本原理。
- 了解 MPI 程序的 GDB 调试流程。

2 实验内容

- 使用 MPI 点对点通信实现并行矩阵乘法。
- 设置进程数量（1~16）及矩阵规模（128~2048）。
- 根据运行时间，分析程序的并行性能。

3 实验结果

3.1 性能分析

根据运行结果，填入下表以记录不同进程数和矩阵规模下的运行时间：

进程数	128	256	512	1024	2048
1	0.005997 s	0.024128 s	0.444456 s	3.531901 s	48.244504 s
2	0.003672 s	0.013877 s	0.211537 s	1.776948 s	27.174872 s
4	0.001991 s	0.009174 s	0.133641 s	0.912547 s	18.904544 s
9	0.000955 s	0.004348 s	0.081890 s	0.489648 s	8.344542 s
16	0.000677 s	0.002918 s	0.053049 s	0.319259 s	6.033024 s

表 1: 不同进程数和矩阵规模下的运行时间

3.2 结果分析

本节使用常用并行性能指标：加速比 $S_p = T_1/T_p$ 和并行效率 $E_p = S_p/p$，其中 T_p 表示 p 个进程下的运行时间。基于 C 实现的完整基准结果，得到以下定量观察：

- 在不同矩阵规模下的结果：
  - $N = 128$：$T_1 = 0.005997\,\mathrm{s}$，$T_{16} = 0.000677\,\mathrm{s}$，$S_{16} \approx 8.86$，$E_{16} \approx 0.55$.
  - $N = 512$：$T_1 = 0.444456\,\mathrm{s}$，$T_{16} = 0.053049\,\mathrm{s}$，$S_{16} \approx 8.38$，$E_{16} \approx 0.52$.

- $N = 2048$：$T_1 = 48.244504\,\mathrm{s}$，$T_{16} = 6.033024\,\mathrm{s}$，$S_{16} \approx 8.00$，$E_{16} \approx 0.50$.

- 结论要点：
  - 随着矩阵规模增大，计算/通信比增大，整体并行效率有明显提升，说明大规模问题更能掩盖通信开销。
  - 对于中等规模（如 $N = 512, 1024$），仍能观察到较好的扩展性；而对于非常小的问题（如 $N = 128$），通信与调度开销占比高，效率波动较大。
  - 未能得到理想的线性加速（$S_p = p$），主要受通信、内存带宽和负载划分影响。

4 讨论题

4.1 讨论题答案

- 在内存受限情况下，如何进行大规模矩阵乘法计算?

  回答：采用分块（block）方法与外存/流式计算相结合。将矩阵按行或列分块，逐块读取并进行局部乘加，避免一次性载入整个矩阵；使用内存映射（mmap）或将数据分片存储在并行文件系统上；在分布式环境下可采用行/列分片并利用 MPI 分发与汇总，必要时使用压缩表示或低秩近似减少内存占用。

- 如何提高大规模稀疏矩阵乘法性能?

  回答：使用稀疏矩阵专用存储和算法（CSR/CSC/COO）；按非零模式划分任务以减少通信与负载不均；利用缓存友好的分块与集合并（blocking）、并行化（多线程 + 分布式）；在乘法中优先跳过零值，使用稀疏 BLAS 或图计算库（例如 SuiteSparse、Intel MKL Sparse、PETSc）；对于非常大的稀疏问题，可尝试近似算法或图划分来减少跨进程通信。

4.2 讨论题实验

4.2.1 分块（外存）实验

模式	N	block	平均时间 (s)	峰值内存 (MB)
naive	512	64	0.471577	22.95
blocked	512	64	0.076163	24.44
naive	512	128	0.462930	23.64
blocked	512	128	0.078401	22.81
naive	1024	64	3.400359	46.24
blocked	1024	64	0.621179	48.40
naive	1024	128	3.434431	43.89
blocked	1024	128	0.618176	46.93

表 2: 分块（外存）实验结果

分块方法对比 下表为对比表，计算了每组配置下的加速比（naive/blocked）以及峰值内存差值（blocked - naive）：

N	block	naive(s)	blocked(s)	speedup	mem diff (MB)
512	64	0.471577	0.076163	6.19	+1.49
512	128	0.462930	0.078401	5.91	-0.83
1024	64	3.400359	0.621179	5.48	+2.16
1024	128	3.434431	0.618176	5.56	+3.04

表 3: block 方法与 naive 方法对比（速度比 = naive / blocked）

分块实验分析

- 在给定的四组配置中，分块实现对比天真实现的速度提升在约 5.48 $\sim$ 6.19×，平均约 5.8×，说明分块策略显著提高了缓存局部性并降低了内存带宽压力。
- 峰值内存差异在数 MB 范围（从约 -0.83 MB 到 +3.04 MB 不等），表明分块实现有时会增加临时缓冲开销但总体内存变化有限。
- 对比不同块大小（64 vs 128），对性能影响有限：在部分配置中 128 略优，实际最优块大小仍需根据硬件缓存层次调优。

4.2.2 稀疏矩阵实验

N	density	平均时间 (s)	峰值内存 (MB)
1000	0.01	0.006954	21.45
1000	0.001	0.001011	22.96
2000	0.01	0.064032	46.88
2000	0.001	0.008670	45.41

表 4: 稀疏矩阵实验结果

稀疏实验分析

- 对于 $N = 1000$，密度从 0.001 增至 0.01，时间从 0.001011 s 提升到 0.006954 s，约 6.9× 更慢；对于 $N = 2000$，对应比值约 7.4×。这说明计算量近似随非零元素数量线性增长。
- 峰值内存随问题规模 N 明显增加（$N = 1000$ 时约 $21 \sim 23$ MB，$N = 2000$ 时约 $45 \sim 47$ MB），对密度影响较小，表明实现的峰值内存主要受矩阵维度支配。
- 总结：当矩阵高度稀疏（低密度）时，稀疏专用存储与算法能显著降低计算时间；当密度增大，成本近似按非零元素数量线性增长。